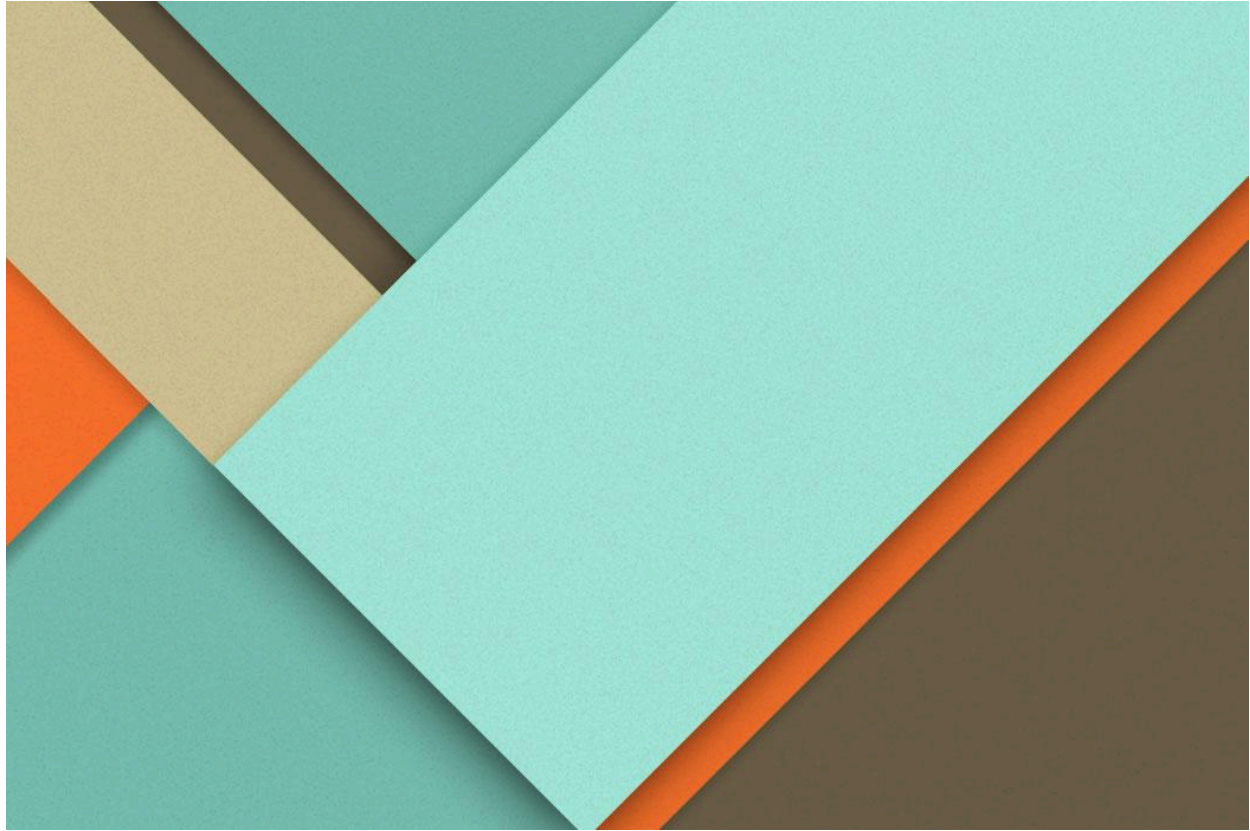




Capstone Two Report

07.07.2025

—

Sadekin Hasnath

California Wildfire Binary Classification Report

Overview

Wildfires are a recurring destructive hazard in California, intensified by climate change, extreme weather, and environmental conditions. The severity of wildfires can be tragic not just to citizens but to the ecosystem as a whole. In this project, I would aim to develop a binary classification model to predict the likelihood of wildfire occurrence (0 / 1) in a specific location and time window based on historical fire records and weather conditions. The ultimate goal of this project would be a valuable tool for the safety of the citizens of California. This tool will allow for early warning systems guidance in order to prepare for possible future wildfires.

Problem Statement

Can we predict whether a wildfire is likely to occur in a given region of California based on environmental weather patterns and historical data? Can weather features such as temperature, humidity, or categorical data such as the seasons, along with the temporal location of California. Is it used to predict the outcome? All these data combined with past wildfire records are used to build a model to predict the likelihood of future wildfire events occurring with high accuracy to prepare for early safety measures, just like we have for weather such as Thunderstorms, tornadoes, floods, etc.

Data Wrangling

For this project, the data wrangling was a crucial one; this notebook took me the most time to complete and debug. I had to roll over one of the steps to EDA due to the notebook becoming longer than usual. First and foremost, we had a big dataset, initially we had 1635 rows by 40 columns.

```
wildfire_data.info()

[3]

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 1636 entries, 0 to 1635
Data columns (total 40 columns):
#   Column                                Non-Null Count  Dtype
---  -
---
```

However, a lot of these rows were not necessary in order for the main objective of this classification project. The goal was to find all the features that occurred preceding a wildfire event such as "Started", temporal ones like "latitude/longitude" all relevant for modeling to predict a target variable that we initialized called "Wildfire Occurred" set to 1 for real wildfire event and to enable with the binary classification. We didn't need any post information such as how the fire was extinguished, the firefighters deployed, these are additional info that wouldn't necessarily help with predicting the target variable during modeling. After keeping the essential columns, I had to handle all rows with missing data and rows with invalid geolocation (Latitude, Longitude). Proceeded to remove duplicate data, there were almost 51 records with identical Location & Started columns, so we subset the data to keep unique columns.

```
print("Before:", len(wildfire_data))

Before: 1636

wildfire_data = wildfire_data.drop_duplicates(
    subset=['Latitude', 'Longitude', 'Started', 'ArchiveYear', 'AcresBurned', 'Location'],
    keep='first'
)

print("After:", len(wildfire_data))

After: 1609
```

Also mapped the counties into 4 broader categories to make it easier to draw insights based on the data, rather than draw insights from 60 raw counties.

The regions will be as follows:

- Northern California
- Central California
- Southern California
- Western California

We created a dictionary to map each county into one of these four regions. A new column, called **'Region'**, will help categorize counties rather than treating every county separately, although we can definitely run analysis on that as well.

```
county_to_region = {
    # Northern California
    'Del Norte': 'Northern California', 'Humboldt': 'Northern California', 'Mendocino': 'Northern California',
    'Lake': 'Northern California', 'Trinity': 'Northern California', 'Siskiyou': 'Northern California',
    'Shasta': 'Northern California', 'Tehama': 'Northern California', 'Lassen': 'Northern California',
    'Plumas': 'Northern California', 'Modoc': 'Northern California', 'Nevada': 'Northern California',
    'Butte': 'Northern California', 'Glenn': 'Northern California',

    # Central California
    'Yolo': 'Central California', 'Sutter': 'Central California', 'Colusa': 'Central California',
    'Yuba': 'Central California', 'Placer': 'Central California', 'El Dorado': 'Central California',
    'Sierra': 'Central California', 'Alpine': 'Central California', 'Amador': 'Central California',
    'Calaveras': 'Central California', 'Tuolumne': 'Central California', 'Mariposa': 'Central California',
    'Mono': 'Central California', 'Stanislaus': 'Central California', 'San Joaquin': 'Central California',
    'Merced': 'Central California', 'Madura': 'Central California', 'Esmeralda': 'Central California'
```

The next crucial step was to simulate non wildfire events, since the dataset contained only real historical wildfire records, we didn't have data for any non-wildfire event for the project to be truly a binary classification. Therefore, using our dataset, we created a new simulated dataset and ensured that it didn't align with the dates of actual wildfire events and had equal entries to match, and in order to make it true to the nature of wildfire prediction. We started with an original dataset of 1635 entries, but ended with a total of 2864 entries with non-wildfire events (0) and actual wildfire events (1)

```
merged_data.head()
```

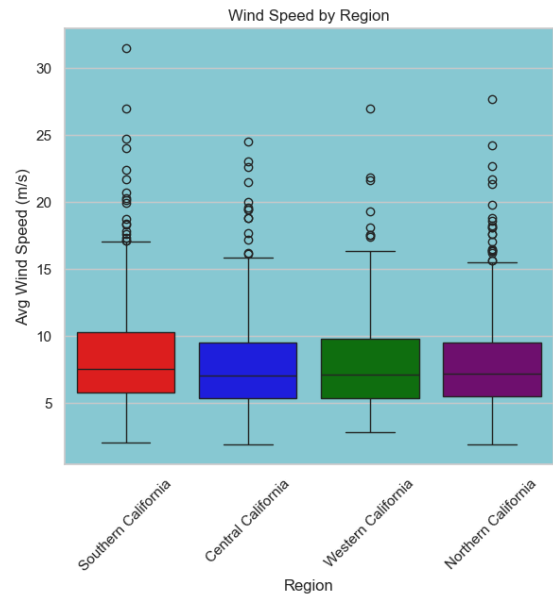
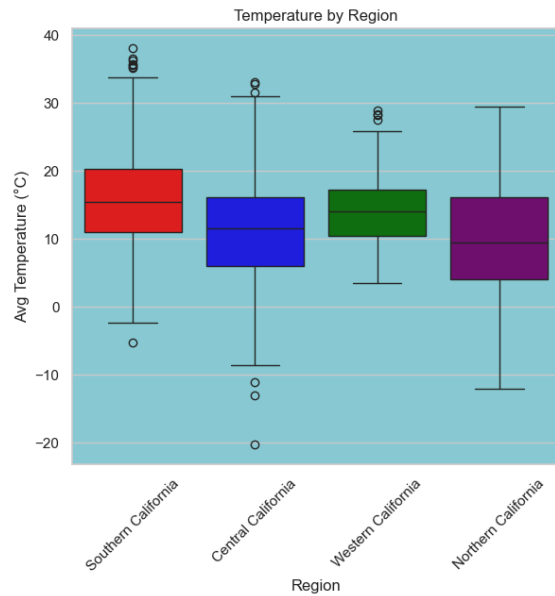
	Latitude	Longitude	ArchiveYear	Started	AcresBurned	Counties	WildfireOccurred	Region	Date
0	40.785933	-121.017469	2013	2013-01-01 08:14:00+00:00	0.0	Lassen	0	Northern California	2013-01-01
1	33.469231	-115.114638	2013	2013-01-02 16:51:00+00:00	0.0	Riverside	0	Southern California	2013-01-02
2	40.816333	-121.838142	2013	2013-01-04 15:07:00+00:00	0.0	Shasta	0	Northern California	2013-01-04
3	34.186179	-118.661168	2013	2013-01-06 17:30:00+00:00	0.0	Los Angeles	0	Southern California	2013-01-06
4	37.507826	-122.105244	2013	2013-01-08 05:47:00+00:00	0.0	Alameda	0	Western California	2013-01-08

Exploratory Data Analysis (EDA)

In this notebook, we added a categorical season column to draw seasonal wildfire trends. The first crucial step in this notebook was to fetch the weather records, since for this notebook, it was to find actual insights rather than code and fix up the data. Using the Open-Meteo Weather API, we fetched weather records for each entry, such as the temperature, humidity, wind speed, and precipitation. There were a lot of things that we learned from the graphs using these features. I learned something using the API, as it was my first time; it took exactly one hour to fetch all the data for all the records. I found that to be interesting.

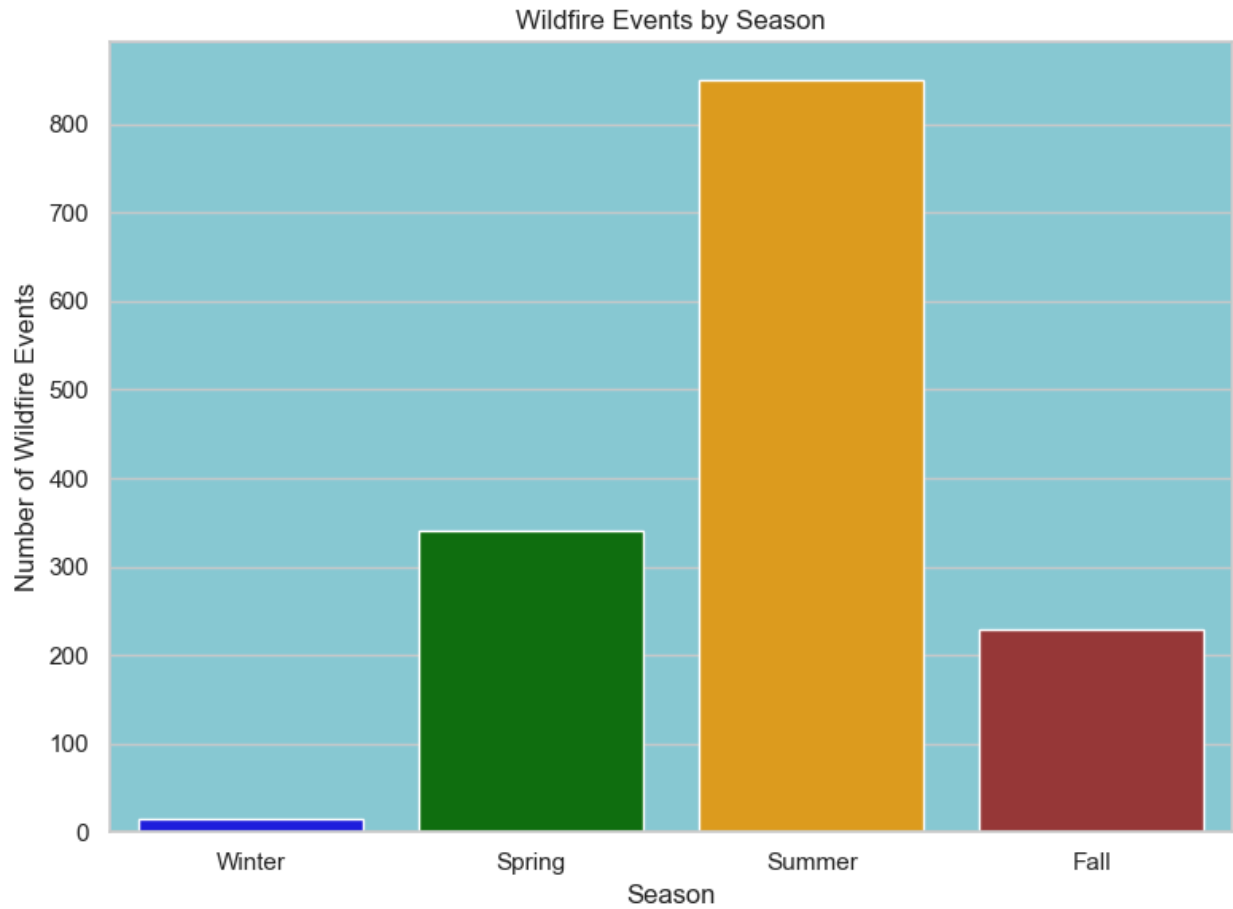
Temperature	Humidity	WindSpeed	Precipitation
21.7	62.0	5.3	0.0
14.5	68.0	6.0	0.0
18.9	53.0	13.9	0.0
15.6	22.0	10.6	0.0
16.4	32.0	5.8	0.0

From wildfire trends, we first started with Regional Trends, with actual wildfire and non-wildfire events. The findings were that Southern California accounted for the largest share of wildfires, 33.5%, slightly ahead of Northern California, 28.9%. However, average temperature and wind speed distributions across regions were fairly similar. One notable difference was that Southern California had more wind speed outliers, which could explain its elevated wildfire rate despite only modest differences in median temperature, which could show that even such a little difference is not due to chance. However, humidity and precipitation distributions were not strongly associated with region-specific fire patterns.

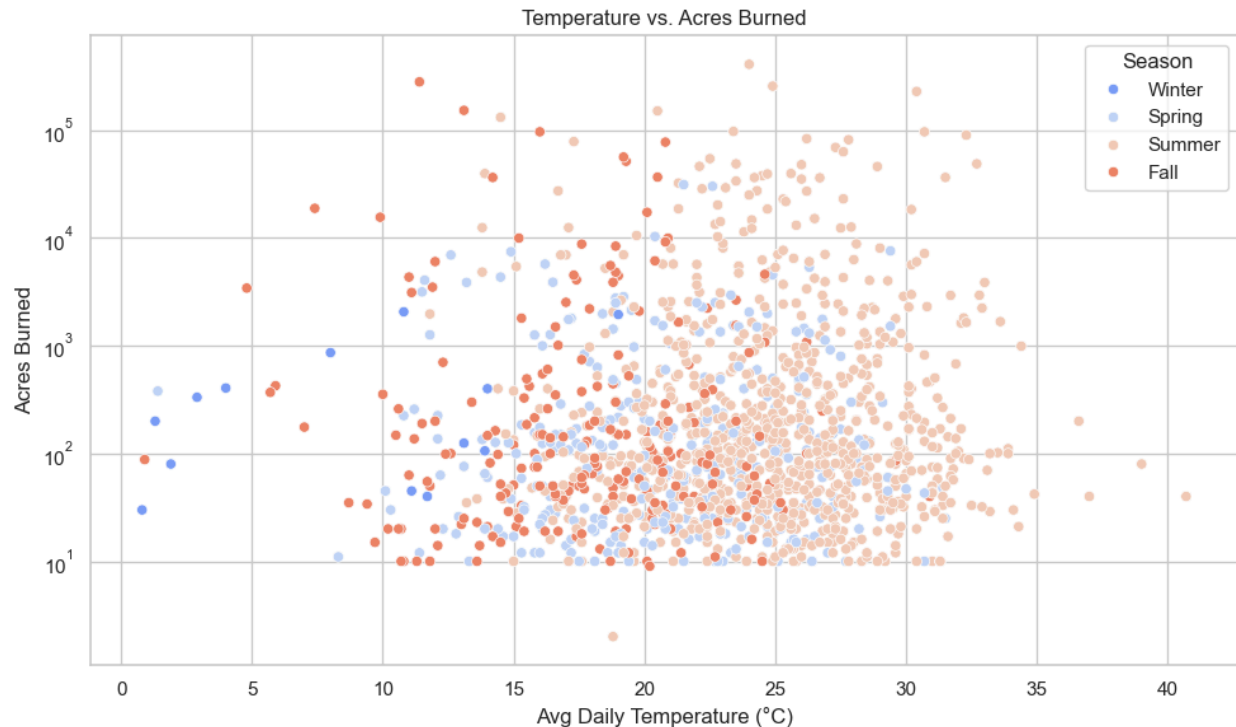


Compared to Real vs. Simulated (Non-Wildfire) Events, the temperature and wind speed are generally higher on wildfire days, especially in Southern California. Found that non-wildfire days had similar weather patterns, but lacked the extreme values seen on wildfire days.

By region, it was not much of a surprise to find that almost 800+ wildfires occurred during summer, which led to more findings showing that temperature was the strongest predictor feature.



Used scatterplots to find the relationship between temperature and acres burned, found that higher temperatures were associated with larger fires, especially in summer, where the temperature is much higher.



The second most important predictor was humidity, which was negatively correlated with temperature, so low humidity and high temperature had the greatest chance for a wildfire occurring. Overall, EDA provided a lot of insights, and all of it would not have been possible with just the original dataset, combining the dataset, and manually creating new columns to create more insights. Although we were limited to strictly weather patterns in this project, in the future combining other wildfire events and factors such as vegetation or population density would be good to explore.

Model Preprocessing & Training

This notebook was rather short due to mainly preprocessing for the modelling notebook. The first thing we had to handle in this notebook was to convert the Region and Season columns into numerical dummy variables by using one-hot encoding. Dropped the first category to avoid multicollinearity. After that, we scaled the numerical features. (Temperature, Humidity, WindSpeed, Precipitation) using StandardScaler so that all features are on the same scale.

	Temperature	Humidity	WindSpeed	Precipitation
975	-1.65	0.94	-0.80	-0.22
1854	-0.25	-0.05	-0.51	-0.22
1565	0.33	-0.53	1.07	-0.22
2517	-0.20	-0.48	-0.70	-0.22
1408	-1.16	1.08	1.21	0.23

This would prevent larger ranges from dominating the model, such as humidity against precipitation. After that, we split the data into Training and Testing sets using a 70/30 split, stratified, even though the dataset wasn't imbalanced, but this ensures it is further.

```
# Define feature matrix X and target vector y
X = wildfire_en.drop(columns=['WildfireOccurred', 'Started', 'Date', 'Counties', 'ArchiveYear'])
y = wildfire_en['WildfireOccurred']

# Split the data into training and testing sets (70/30 split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Display the shape of the resulting splits
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (2004, 13)
X_test shape: (860, 13)
y_train shape: (2004,)
y_test shape: (860,)
```

Modelling

In the final notebook of the wildfire binary classification project, the goal was to assess the performance of different models for classification types to get the best metric score possible, focusing on recall. I decided to test multiple models on this classification project to see which model would be the best for this type of classification project. Specifically, the goal was to find the best possible recall score, as this was the most important for predicting actual wildfire (True positives) events given the data. The second important metric was precision; this was important for reducing a negative alarm (false negatives).

Logistic Regression Model

First, to get a check on the project, we started with the simplest model, Logistic Regression. This was just to check if a model can truly predict on the test set from the test train split step we did earlier. At first, we had a prediction of 99% near a perfect score, but it was due to the fact that I forgot to reduce the AcresBurned column. Lowest performing model, did not tune to any param grid for the best estimators.

Logistic Regression Model Evaluation

- Accuracy: 0.81
- Precision: 0.79
- Recall: 0.84
- F1 Score: 0.81

Random Forest Model

Then, moving on to the next, a slightly more complicated model with hyperparameter tuning was Random Forest. First, by setting the max depth to 3 to prevent the model from overfitting, we had a model performance as shown below. This model was slightly better at predicting true positives, but performance dropped for precision and false positives.

```
[94] ... Accuracy: 0.8069767441860465  
Precision: 0.7754677754677755  
Recall: 0.8654292343387471  
F1 Score: 0.8179824561403509
```

After using GridSearch for hyperparameter tuning using the best estimator, the model slightly improved. The model performance is now sitting at 88% recall for predicting true positives and almost at 80% for predicting false positives. This was a slight improvement, and our model is already performing very well.

```
[31] ... Accuracy: 0.8197674418604651  
Precision: 0.7875  
Recall: 0.877030162412993  
F1 Score: 0.8298572996706916
```

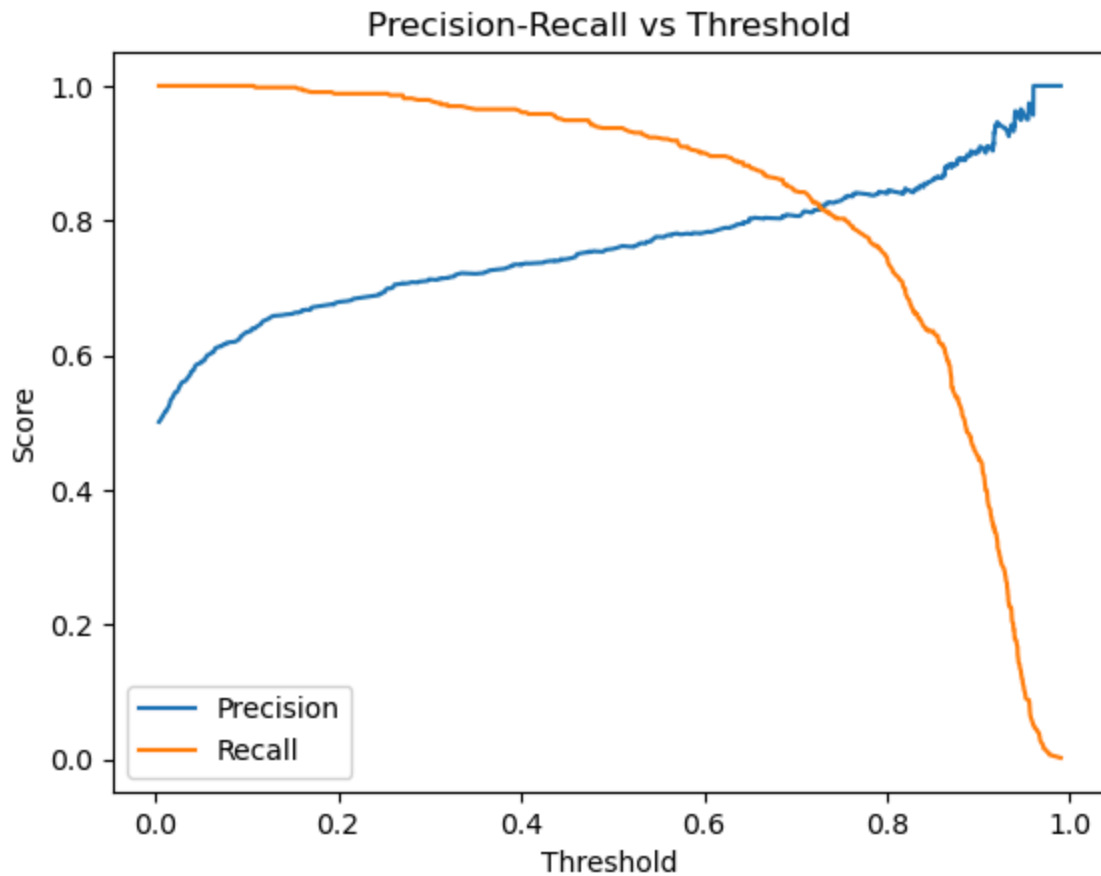
XGBoost Model

Then, moving on to a more complex model, XGBoost is a highly efficient and flexible gradient boosting algorithm, widely used for structured data tasks. It builds an ensemble of decision trees in a sequential manner, optimizing for both speed and performance. In this notebook, we fit a baseline XGBoost model, evaluate its performance, and then perform a hyperparameter search to balance precision and recall using the F1 score.

This was the performance using a predefined param grid for the XGBoost model and the upcoming LightGBM model. This performance is good, but it has a terrible precision (false positive), regardless of the very high recall of 94%, which would be costly to deploy units for a false alarm.

```
Accuracy: 0.8186046511627907  
Precision: 0.7579737335834896  
Recall: 0.9373549883990719  
F1 Score: 0.8381742738589212
```

I had to set a threshold in order to balance the score by setting it to 0.58. The model would need a much higher score in order to classify it as a true positive wildfire event; this was a necessary trade-off. With this model we got the best recall score of 91% for predicting true positives and 78% for false positives.



LightGBM Model

Moving on to the LightGBM model, which is another powerful gradient boosting algorithm, often faster and more efficient than XGBoost.

Here, we fit a baseline LightGBM model and evaluate its performance. It did not perform that well, although overall, for all the metrics, it was a good model to choose.

```
Accuracy: 0.8313953488372093
Precision: 0.8177777777777778
Recall: 0.8538283062645011
F1 Score: 0.8354143019296254
```

Using hyperparameter tuning with cross-validation, we achieved a slightly better performance metric; however, it is not a high average since Random Forest got 88% for recall.

Accuracy: 0.8348837209302326
Precision: 0.8120950323974082
Recall: 0.8723897911832946
F1 Score: 0.8411633109619687

Overall, this model has similar performance to XGBoost, with a recall of 0.87 and ROC AUC of 0.91 on the test set. Although this model performed well, it does not beat the XGBoost model, which performed 0.87 on a cross-validation set, but on the test set it performed. Overall, the metrics are very similar to Random Forest therefore, this model is the second best in performance if we factor in all the metrics.

Model Performance

Model	Logistic Regression	Random Forest	XGBoost	LightGBM
Accuracy	0.81	0.82	0.83	0.83
Precision	0.79	0.79	0.78	0.81
Recall	0.84	0.88	0.91	0.87
F1 Score	0.81	0.83	0.83	0.84